

In-Class Exercise/Demo



In-Class Demo

- Let's practice the [DOM](#) and work with [DOM APIs](#).
- First, create a new github repository [m5-w4-d1-demo](#) and clone this in the WESTCLIFF folder.
- Then head over to GAP and download [m5-w4-d1-demo.zip](#). Unzip and drop the content in the clone folder.
- Launch VS Code and navigate to the clone folder.
- Open [demo1.html](#)



In-Class Demo #1

- Let's review the code
- There is `window.onload` function that is loading when a page loads

```
window.onload = function()
```

- Next, we have 2 `const` variables setting a “h1” tag and creating a `textnode`.

```
const heading = document.createElement("h1");  
const heading_text = document.createTextNode("Big Head!");
```

- Let's open the file in the “Live Preview”. You will not see anything on the page. Let's review the code on the page, there is nothing added to body.
- This is because we have not created any nodes in the DOM. Let's create a node in DOM and add it to body.



In-Class Demo

- Let's add after creating heading_text

```
heading.appendChild(heading_text);
```

- Next let's add the heading to document.

```
document.body.appendChild(heading);
```

Complete Code

```
window.onload = function() {  
    const heading = document.createElement("h1");  
    const heading_text = document.createTextNode("Big Head!");  
    heading.appendChild(heading_text);  
    document.body.appendChild(heading);  
}
```



In-Class Demo

Previewing on the browser:

Big Head!

Inspect the code. You will see the `<h1>` element is now created.

```
<html> event
  <head>
    <script>
      // run this function when the document is loaded window.onload = function() { // create a couple of elements in
      an otherwise empty HTML page const heading = document.createElement("h1"); const heading_text =
      document.createTextNode("Big Head!"); heading.appendChild(heading_text); document.body.appendChild(heading); }
    </script>
  </head>
  <body data-new-gr-c-s-check-loaded="8.874.0" data-gr-ext-installed="">
    <h1>Big Head!</h1>
  </body>
</html>
```



In-Class Demo #2

- Open [demo2.html](#) and review the code.
- There is a form with a few different `<select>` tags, a [link](#) and an `input` button.
- If you notice there is nothing in the `<form>` tag, action or other attributes
- Let's open the document in browser, using live preview
- You will notice that the drop downs do not do anything. We would like the [text to change](#) when [black](#) or [red](#) is selected.
- Next we would like the [background color](#) to change when [white](#) or [gray](#) is selected
- Next we would like the [link color](#) to change when [blue](#) or [green](#) is selected.



In-Class Demo #2

- Let's update the first drop down. We will add onChange attribute to <select>

```
<select onChange="setBodyAttr('text', this.options[this.selectedIndex].value);">
```

- Next let's write the `setBodyAttr` function

This function will take the `attr` and `value` and update the `document.body`

Let's add this code to <head> after the title in a <script> tag

```
function setBodyAttr(attr, value) {  
    (document.body) document.body[attr] = value;  
}
```



In-Class Demo #2

- We'll add an if statement in case the function is unable to create a attribute in DOM.

```
<script>
  function setBodyAttr(attr, value) {
    if (document.body) document.body[attr] = value;
    else throw new Error("no support");
  }
</script>
```

- Preview this on the browser.



In-Class Demo #2

Preview on the browser

Inspect elements in dev tools:

text

bgColor

link
 [\(sample link\)](#)

```
<html>
  <head>...</head>
  <body data-new-gr-c-s-check-loaded="8.874.0" data-gr-ext-installed="" cz-shortcut-listen="true" text="black">
    <div style="margin: .5in; height: 400px;">
      <p>
        <b>
          <code>text</code>
        </b>
      </p>
      <form>
        <select onchange="setBodyAttr('text', this.options[this.selectedIndex].value);">...</select> event
        <p>...</p>
        <select>...</select>
      </form>
    </div>
  </body>
</html>
```



In-Class Demo #2

Change the color to red and check elements again:

text

red

bgColor

white

link

blue (sample link)

version

```
<html>
  <head> ... </head>
  <body data-new-gr-c-s-check-loaded="8.874.0" data-gr-ext-installed="" cz-shortcut-listen="true" text="red">
    <div style="margin: .5in; height: 400px;">
      <p>
        <b>
          <code>text</code>
        </b>
      </p>
      <form>
        <select onchange="setBodyAttr('text', this.options[this.selectedIndex].value);"> ... </select> event
        <p> ... </p>
        <select> ... </select>
      </form>
    </div>
  </body>
</html>
```



In-Class Demo #2

- Let's continue add an `onChange` attribute:

```
<p><b><code>bgColor</code></b></p>
<select onChange="setBodyAttr('bgColor',
    this.options[this.selectedIndex].value);">
    <option value="white">white</option>
    <option value="lightgrey">gray</option>
</select>
<p><b><code>link</code></b></p>
<select onChange="setBodyAttr('link',
    this.options[this.selectedIndex].value);">
    <option value="blue">blue</option>
    <option value="green">green</option>
</select>
<small>
    <a href="http://some.website.tld/page.html" id="sample">
        (sample link)
    </a>
</small><br />
```



In-Class Demo #2

- Test it on your browser and inspect elements.

text

red

bgColor

gray

link

green (sample link)

version



In-Class Demo #3

- Open [demo3/index.html](#) and review the code
- We are using [React](#), [ReactDOM](#) and [Babel CDN](#)
- The goal for this exercise is to display an [associated ID](#) when an element is clicked.
- Let's open [on-click.js](#)
- We have a [OnClickElements](#) component with an empty constructor and super
- We then have a [render](#) that is [returning](#) many HTML elements.
- At the bottom, we are using [ReactDOM](#) to render the element



In-Class Demo #3

- First, add an `onClick` to return elements so we can display an ID.

```
<div id="div-element" onClick={(e) => this.handleClick(e)}>I am DIV</div>
```

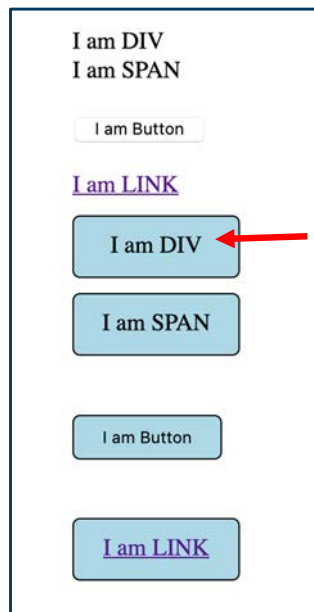
- Next create a `handleClick` method after the constructor:

```
    }  
    handleClick(event) {  
        alert(`hey! you clicked: ${event.target.id}`);  
    }  
    render() {
```

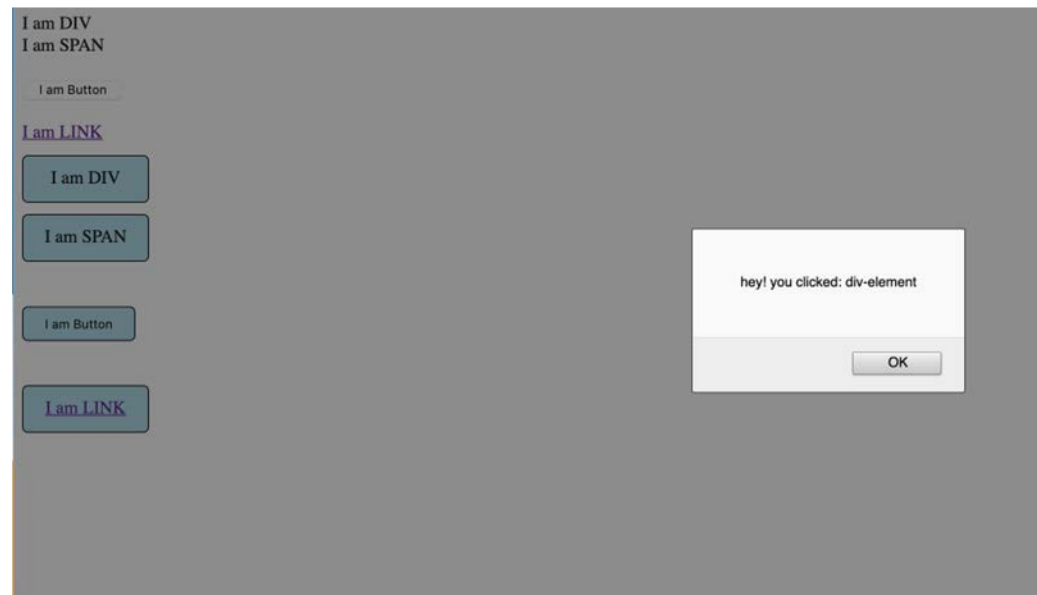


In-Class Demo #3

- Let's review in the browser



Click on "I am DIV"





In-Class Demo #3

- Let's add the same `onClick` code to rest of the elements and test our code.
- You should be able to see the corresponding ID when an element is clicked.

```
return (  
  <div className="container-div">  
    <div id="div-element" onClick={(e) => this.handleClick(e)}>  
      I am DIV  
    </div>  
  
    <span id="span-element" onClick={(e) => this.handleClick(e)}>  
      I am SPAN  
    </span>  
    <br/>  
  
    <button id="button-element" onClick={(e) => this.handleClick(e)}>  
      I am Button  
    </button>  
    <br/>  
  
    <a id="link-element" href="" onClick={(e) => this.handleClick(e)}>  
      I am LINK  
    </a>  
  
    <div  
      id="div-element-2"  
      className="button"  
      onClick={(e) => this.handleClick(e)}  
    >  
      I am DIV  
    </div>  
  
    <span  
      id="span-element-2"  
      className="button"  
      onClick={(e) => this.handleClick(e)}  
    >  
      I am SPAN  
    </span>  
    <br/>  
  
    <button  
      id="button-element-2"  
      className="button"  
      onClick={(e) => this.handleClick(e)}  
    >  
      I am Button  
    </button>  
    <br/>  
  
    <a  
      id="link-element-2"  
      className="button"  
      href=""  
      onClick={(e) => this.handleClick(e)}  
    >  
      I am LINK  
    </a>  
  </div>  
)  
);
```




In-Class Exercise Demo

- **DUE:**

Today 10.30PM PT

- **Submission:**

Post Github URL link on GAP Week 3 Day 1 Exercise